AFTER THE MOBILITY – APRÈS LA MOBILITÉ

Table D – Traineeship Certificate by the Receiving Organisation

Tableau D – Attestation de stage de l'organisme d'accueil

Name of the trainee / Nom du stagiaire: Hamza ARIOUICH

Name of the Receiving Organisation: Universitat Politècnica de València (UPV)

Sector of activity: Computer Engineering

Address: Universitat Politècnica de València, Camí de Vera s/n, Valencia, 46022, Spain – `https://www.upv.es/`

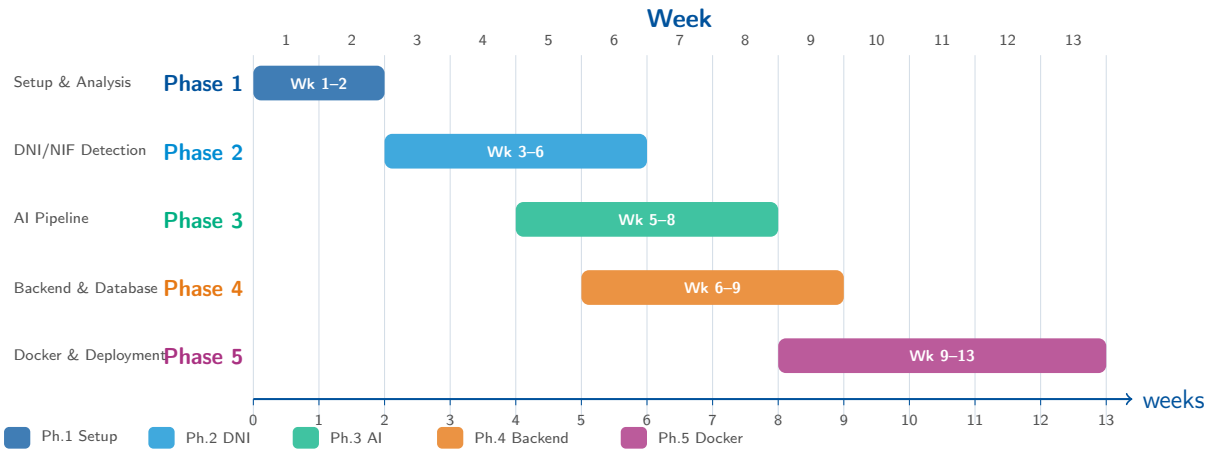
Start / end of complete traineeship: 08/04/2026 – 30/07/2026

Start / end of physical mobility: 08/04/2026 – 23/07/2026

Traineeship title: Integration of a Handwritten Number Recognizer into a Web Application for Exam Management

---

Detailed Programme of the Traineeship Period



The trainee worked on **Madrina**, a full-stack web application that enables university teachers to import, split, automatically recognise, grade, and redistribute scanned exam papers. The application relies on two convolutional neural-network (CNN) models: one for reading handwritten DNI/NIF identity-number characters and one for recognising handwritten grade digits.

Phase 1 – Codebase analysis and environment setup (weeks 1–2).

- Studied the existing Python AI pipeline, the Node.js/Express backend, and the Angular 13 frontend.
- Configured the local development environment: Python virtual environment, MySQL schema (`MargotDB`), Node.js dependencies, and Angular CLI toolchain.
- Reviewed the two PyTorch checkpoints (`best_model.pt` for DNI recognition, `exammodel4.pt` for grade detection) and their integration as child processes of the backend.

Phase 2 – DNI/NIF detection improvements (weeks 3–6).

- Analysed failure cases in the existing DNI region-of-interest extraction module (`utils/traitement/dni_processing.py`).
- Reworked the character-segmentation and bounding-box logic in the coordinate-computation pipeline (`compute_coords_from_main_block`) to reduce false positives on skewed or low-contrast scan areas.
- Improved the Levenshtein-based student-matching logic: tuned the similarity threshold (`MATCH_RATIO_THRESHOLD`) and added validation of the reconstructed identifier before attempting a database lookup.
- Fixed a key-handling bug in the student-roster parsing that caused silent mismatches between OCR output and the `alumnos` table (commit `f25f34f`).
- Fixed a residual marker-detection regression in the copy-boundary detector (`buscar_inicio_copias`), introduced during refactoring (commit `b637bb4`).

Phase 3 – AI model integration and pipeline hardening (weeks 5–8).

- Validated the end-to-end prediction pipeline: PDF ingestion via PyPDF2 and Poppler, image extraction, CNN inference with PyTorch on both CPU and CUDA, and structured result emission over stdout (`progress`, `new-calificado`, `done`).
- Introduced a `validate_runtime()` check at startup to detect missing model checkpoints or malformed student-roster arguments before any page is processed.
- Added parallel preprocessing using `ProcessPoolExecutor` to distribute coordinate-computation work across available CPU cores.
- Translated all in-code comments and documentation from Spanish/French to English (commit `32f17a3`) to improve maintainability for international contributors.

Phase 4 – Backend and database work (weeks 6–9).

- Extended the Node.js/Express Data Access Layer (`DAL.js`) with new SQL queries against the `MargotDB` MySQL schema (tables: `usuarios`, `asignaturas`, `alumnos`, `examenes`, `calificados`).
- Reviewed and validated the session-token authentication interceptor applied to all `/api` routes.
- Contributed to Socket.IO event handling (`examen-predict`, `examen-predict-kill`) to ensure real-time progress updates reach the Angular frontend reliably.
- Verified PDF-to-image conversion orchestration (Poppler `pdftoppm`, `pdfseparate`, `pdfunite`) and ZIP archive generation (`archiver`).

Phase 5 – Dockerisation and production deployment (weeks 9–13).

- Wrote and refined `Dockerfile.backend` and the Angular `Dockerfile` so that the backend (Node.js + Python AI module) and the frontend (Nginx-served static build) run as separate Docker services.
- Authored `docker-compose.yml` (development) and `docker-compose.prod.yml` (production) orchestrating the frontend, backend, and MySQL containers with environment-variable injection and named volumes for persistent data.
- Supported an external database configuration option in `docker-compose` (commit `16faafe`), allowing the application to connect to an existing MySQL server rather than spinning up a container.

- Wrote comprehensive deployment documentation (`README_deploy.md`, `README_deploy_prod.md`): Nginx reverse-proxy configuration, Socket.IO upgrade headers, TLS, systemd service creation, and backup/restore procedures.
- Performed end-to-end validation of the Dockerised stack: account creation, exam upload, AI prediction, manual correction, and ZIP export.

Knowledge, Skills and Competences Acquired

Technical knowledge.

- Deep understanding of convolutional neural networks applied to optical character recognition (handwritten digit and alphanumeric string recognition with PyTorch).
- Knowledge of image-processing techniques: bounding-box extraction, contrast normalisation, region-of-interest cropping with OpenCV and NumPy.
- Understanding of the full software stack of a modern web application: relational database (MySQL), REST/WebSocket backend (Node.js, Express, Socket.IO), and single-page application frontend (Angular, TypeScript, RxJS).
- Practical knowledge of Docker and Docker Compose for multi-container application packaging, Nginx reverse-proxy configuration, and production deployment on Linux servers.

Practical skills.

- Ability to diagnose and fix bugs in an AI inference pipeline by combining log analysis, manual testing with sample PDFs, and controlled unit experiments.
- Proficiency in SQL query writing and optimisation within a Node.js DAL pattern.
- Skill in writing production-grade `docker-compose` configurations including secret management via `.env` files, health checks, and volume mapping.
- Experience writing clear technical documentation in English for an international audience.

Transversal competences.

- Ability to read and extend a pre-existing multi-language, multi-technology codebase.
- Autonomous problem-solving and debugging in an international research environment.
- Version-control discipline: meaningful commit messages, feature branches, and pull-request workflow on GitHub.
- Adaptability to a Spanish-language academic environment and to the working culture of the Universitat Politècnica de València.

Evaluation of the Trainee

The trainee, Hamza ARIOUICH, demonstrated a high level of technical competence and professional commitment throughout the internship period.

- **Quality of work:** The contributions delivered were correct, well-structured, and aligned with the project's coding conventions. Bug fixes were accompanied by root-cause analysis and targeted tests.
- **Autonomy and initiative:** The trainee took ownership of complex tasks (DNI detection improvement, full Dockerisation) with minimal supervision, proposing and implementing solutions independently.

- **Adaptability:** The trainee efficiently navigated a heterogeneous codebase spanning Python, JavaScript/TypeScript, SQL, and shell scripting, and quickly became productive in an unfamiliar academic environment.
- **Communication and documentation:** Technical documentation was produced in clear, professional English. The trainee communicated progress and blockers effectively to the supervising team.
- **Overall assessment:** Excellent. The trainee's contributions materially improved the reliability and deployability of the Madrina application and are integrated into the production branch.

Date: .....

Name, signature and stamp of the Supervisor at the Receiving Organisation:

.....